

Introduction to JavaScript

1. Introduction

What is JavaScript?

JavaScript is a high-level, interpreted programming language originally created by Brendan Eich. It was first developed to add interactivity to web pages in the Netscape browser. Today it is one of the three core technologies of web development alongside HTML and CSS.

Where JavaScript Runs

- **Browser (Client-side):** Handles interactivity, DOM updates, form validation, and dynamic content.
- **Node.js (Server-side):** Used for backend applications, APIs, and server logic.
- **Other platforms:** Through frameworks it can also run on desktop, mobile, and IoT devices.

Role in Web Development

- **HTML** provides the structure (headings, paragraphs, buttons, forms).
 - **CSS** handles presentation (colors, layout, animations).
 - **JavaScript** adds behavior (responding to clicks, validating forms, updating content dynamically).
-

2. Basics of JavaScript

Variables

JavaScript has three main ways to declare variables:

Keyword	Introduced	Can be redeclared	Can be reassigned	Scope
<code>var</code>	Early JS	Yes	Yes	Function scope
<code>let</code>	ES6 (2015)	No	Yes	Block scope
<code>const</code>	ES6 (2015)	No	No	Block scope

Recommended practice

1. Always declare variables before using them.
2. Prefer `const` when the value (or the binding) should not change.
3. Use `let` only when reassignment is required.
4. Avoid `var` unless you must support very old browsers.

Data Types

- **String:** Text values (e.g. `"Hello"`)
- **Number:** Integers and floating-point numbers
- **Boolean:** `true` or `false`
- **Null:** Intentionally empty value
- **Undefined:** Variable declared but not assigned a value
- **Object:** Collection of key-value pairs
- **Array:** Ordered list of values (a special type of object)

Operators

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Comparison:** `==`, `===`, `!=`, `!==`, `<`, `>`, `<=`, `>=`
- **Logical:** `&&` (AND), `||` (OR), `!` (NOT)
- **Assignment:** `=`, `+=`, `-=`, `*=`, `/=`

Note: Prefer strict equality (`===` and `!==`) to avoid type-coercion surprises.

3. Control Flow

Conditional Statements

- `if` – executes a block when a condition is true
- `else` – executes when the condition is false
- `else if` – tests additional conditions
- `switch` – selects one of many code blocks based on a value

Loops

- `for` – repeats a block a fixed number of times
 - `for...in` – iterates over the enumerable properties of an object
 - `for...of` – iterates over the values of an iterable (arrays, strings, etc.)
 - `while` – repeats while a condition remains true
 - `do...while` – similar to `while`, but always runs the block at least once
-

4. Functions

A function is a reusable block of code that performs a specific task. It runs only when it is called (invoked).

Important Concepts

- **Parameters:** Input values received by the function
- **Return value:** The result the function sends back
- **Scope:**
 - Global – accessible from anywhere
 - Local (function) – accessible only inside the function
 - Block – accessible only inside `{ }` when using `let / const`

Types of Functions

1. **Function Declaration** – Named function defined with the `function` keyword. Hoisted (can be called before its definition).

2. **Function Expression** – Function stored in a variable. Not hoisted.
 3. **Arrow Function (ES6)** – Compact syntax using `=>`. Does not have its own `this` binding.
 4. **Anonymous Function** – Function without a name (commonly used as a callback).
 5. **IIFE (Immediately Invoked Function Expression)** – Defined and executed immediately.
 6. **Higher-Order Function** – Takes another function as an argument or returns a function.
 7. **Recursive Function** – Calls itself to solve a problem.
-

5. Arrays

An array is an ordered collection that can hold values of any type (numbers, strings, objects, even other functions).

Common Properties and Methods

- `.length` – number of elements
 - `.push()` – add element(s) at the end
 - `.pop()` – remove the last element
 - `.shift()` – remove the first element
 - `.unshift()` – add element(s) at the beginning
 - `.map()`, `.filter()`, `.reduce()` – powerful methods for transforming and processing data
-

6. Objects

An object is a collection of key-value pairs. Keys are usually strings (or Symbols); values can be any data type, including functions (methods).

Common Operations

- Accessing properties (dot notation or bracket notation)
 - Adding or updating properties
 - Deleting properties
 - Defining methods (functions stored as property values)
 - Using built-in Object methods (`Object.keys()` , `Object.values()` , `Object.entries()` , etc.)
-

7. DOM Manipulation

What is the DOM?

The Document Object Model is a programming interface that represents an HTML or XML document as a tree of nodes (elements, attributes, text nodes, etc.). JavaScript can read and modify this tree to change the page dynamically.

Why the DOM Matters

- Update content (text, images, etc.) without reloading the page
- Respond to user interactions
- Build interactive, dynamic web applications

Typical DOM Tasks

- Selecting elements (`getElementById` , `querySelector` , `querySelectorAll` , etc.)
 - Changing content (`textContent` , `innerHTML`)
 - Changing styles (`style` property or `classList`)
 - Creating and inserting new elements
 - Removing elements
-

8. Event Handling

An event is an action that occurs in the browser (click, key press, mouse movement, form submission, etc.). JavaScript can listen for these events and

execute code in response.

Common Event Types

- `click` – user clicks an element
- `mouseover` / `mouseout` – mouse enters or leaves an element
- `keydown` / `keyup` – key is pressed or released
- `submit` – form is submitted
- `change` – value of an input changes

Events can be attached using HTML attributes, the `addEventListener` method, or older property-based handlers.

9. Callbacks

A callback is a function passed as an argument to another function. The receiving function can call the callback later, often after an asynchronous operation finishes.

Why Callbacks Are Useful

- **Reusability** – the same function can accept different callbacks
- **Asynchronous handling** – useful for operations that take time (file reading, network requests)
- **Control flow** – define what should happen after a task completes

Deeply nested callbacks can lead to “callback hell,” which is one reason Promises and `async/await` were introduced.

10. Promises

A Promise is an object that represents the eventual result of an asynchronous operation. It can be in one of three states:

- **Pending** – the operation is still in progress
- **Fulfilled (Resolved)** – the operation completed successfully
- **Rejected** – the operation failed

Benefits of Promises

- Avoid deeply nested callbacks
 - Cleaner chaining with `.then()` and error handling with `.catch()`
 - Allow sequential asynchronous operations to be expressed more clearly
-

11. Async and Await

`async` / `await` is syntactic sugar built on top of Promises. It lets asynchronous code look and behave more like synchronous code.

- An `async` function always returns a Promise.
- The `await` keyword pauses execution of the async function until the Promise settles (fulfills or rejects).

Advantages

- Code reads top-to-bottom like normal sequential instructions
 - Avoids long `.then()` chains
 - Error handling can use familiar `try...catch` blocks
 - Considered the modern, preferred way to write asynchronous JavaScript
-

12. Closures

A closure is created when a function is defined inside another function and the inner function retains access to the outer function's variables even after the outer function has finished executing.

Why Closures Are Useful

- **Data privacy** – variables can be hidden from the outside world
- **Encapsulation** – only selected functions can interact with the private data
- **Stateful functions** – useful for counters, caching, and event handlers that need to remember previous values
- Forms the foundation of many functional programming patterns in JavaScript
